

基于 ESP32-C3 与 MLX90640 的 热成像相机系统项目报告

微机接口与技术 / 微机原理期末大作业 | rdfrdream

摘要

本项目围绕低成本热成像检测与网页可视化展开，使用 ESP32-C3 作为主控，连接 MLX90640 32 x 24 红外阵列传感器，通过 I2C 读取温度矩阵，并利用 ESP32-C3 自带 WiFi 建立局域网 Web 服务。浏览器端网页负责热图绘制、色温条显示、数据刷新、演示数据导入、全屏展示和 CSV 导出。项目完成了 PlatformIO 工程、ESP32-C3 固件、SPIFFS 网页资源、本地演示服务器、硬件接线方案、调试接口和文档整理；真实 MLX90640 采集链路已进入实物稳定性验证阶段。本报告重点记录项目背景、总体方案、硬件连接、软件实现、调试过程、现阶段结果和后续改进方向。

项目要素	说明
主控平台	ESP32-C3 开发板，负责 I2C 采集、WiFi AP、HTTP Server 和数据接口
传感器	MLX90640 红外阵列传感器，32 x 24，共 768 个温度点，I2C 地址 0x33
显示方式	手机或电脑连接 ESP32-C3 热点后，通过浏览器访问网页热图
工程环境	VS Code + PlatformIO + ESP-IDF，支持编译、烧录、SPIFFS 上传和串口监视
仓库地址	https://github.com/1909899270h-spec/esp32c3-mlx90640-thermal-camera

1. 项目背景与设计目标

热成像技术能够把物体表面的红外辐射转换为可观察的温度分布图，在电路板发热检测、设备状态观察、实验教学演示和低成本测温装置中具有直观价值。传统热像仪成本较高，且通常作为独立仪器使用，不便于与课程中的单片机接口、无线通信和网页可视化内容结合。ESP32-C3 自带 WiFi/BLE，资源足够运行轻量级 Web Server；MLX90640 提供 32 x 24 阵列温度数据，两者组合后可以构成一个低成本、可展示、可记录的热成像系统。

本项目的设计目标不是单纯点亮一个传感器，而是完成从红外阵列采集到网页显示的完整链路：传感器通过 I2C 向 ESP32-C3 提供温度数据；ESP32-C3 进行数据缓存并提供 HTTP 接口；手机或电脑通过 WiFi 连接到 ESP32-C3，使用浏览器查看热图和温度数据；网页端能够记录数据并导出 CSV，便于后续分析。

- 教学目标：覆盖 I2C 总线、GPIO、嵌入式编译烧录、串口调试、WiFi AP、HTTP 接口和浏览器可视化。
- 工程目标：形成可以演示的硬件样机和软件系统，并将源代码、文档、图片证据和运行方式整理到 GitHub 仓库。
- 展示目标：即使真实传感器临时连接不稳定，也能通过本地演示数据说明网页端和数据链路的完整功能。

2. 系统总体方案

系统采用“传感器采集 - 主控处理 - 无线访问 - 网页显示 - 数据记录”的分层结构。MLX90640 负责红外阵列测温，ESP32-C3 负责 I2C 通信、温度帧缓存、WiFi 热点和 HTTP 服务，浏览器网页负责热图绘制、按钮交互和 CSV 导出。该结构把底层采集和上层显示分开，便于调试和替换硬件。

系统总体数据链路

从红外阵列采样到浏览器热图显示，ESP32-C3 同时承担主控、无线接入和小型 Web Server。



核心接口：GPIO4/SDA、GPIO5/SCL、3.3V、GND；网页访问：AP 模式默认 192.168.4.1。

图 1 系统总体数据链路

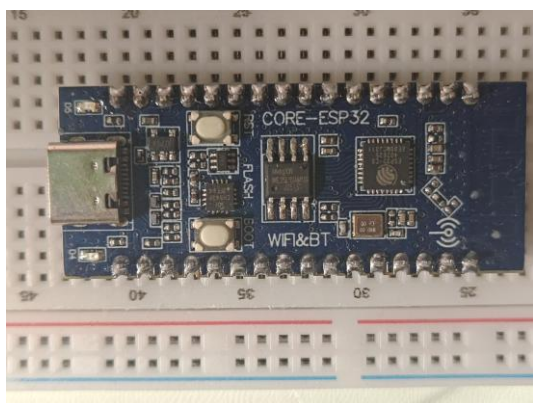
层级	功能	本项目实现
硬件层	供电、传感器连接、I2C 信号	MLX90640 VDD/GND/SDA/SCL 接入 ESP32-C3
采集层	读取红外阵列原始数据并计算温度	src/main.c 调用 MLX90640 API 完成帧读取
网络层	建立无线网络与网页服务	ESP32-C3 AP 模式，默认访问 192.168.4.1
显示层	热图、色温条、数据统计	data/web_script.js 使用 Canvas 绘制热图
记录层	保存时间序列温度数据	网页端记录帧数据并下载 CSV

为什么选择 ESP32-C3

C51 适合基础控制但资源较少，联网和网页显示困难；STM32 控制能力强、外设丰富，但通常需要额外 WiFi 或以太网模块；ESP32-C3 自带 WiFi/BLE，可以同时承担主控、无线通信和小型 Web Server 三个角色，更适合本项目这种低成本、无线化、易展示的热成像系统。

3. 硬件设计与接口连接

硬件样机使用 ESP32-C3 开发板、MLX90640 裸传感器、面包板、杜邦线、5k Ω 上拉电阻和 10uF 电容搭建。选择面包板的原因是便于在调试阶段快速调整接线，但裸传感器引脚较细，临时插接会带来松动和接触不稳定问题，这也是后续真实采集链路需要重点优化的地方。



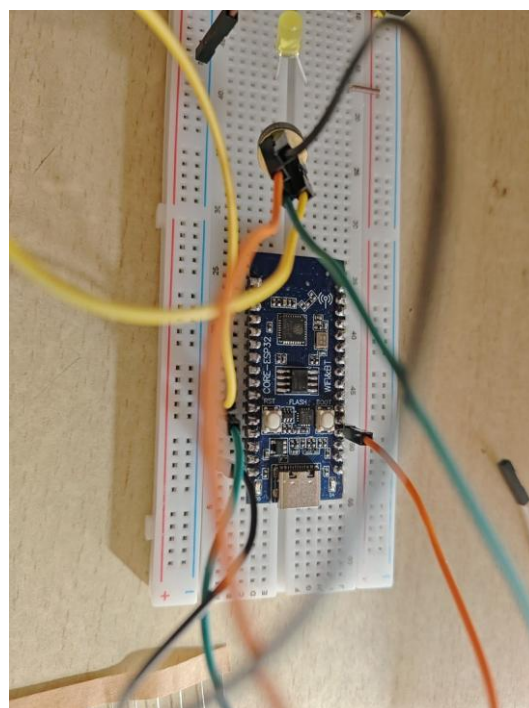
ESP32-C3 开发板正面



ESP32-C3 背面引脚标识



MLX90640 裸传感器



面包板接线样机

3.1 核心接线关系

MLX90640 引脚	ESP32-C3 引脚	作用	说明
VDD	3.3V	传感器供电	避免接 5V，减少烧坏风险
GND	GND	公共地	主控和传感器必须共地
SDA	GPIO4	I2C 数据线	建议通过 5kΩ 电阻上拉到 3.3V
SCL	GPIO5	I2C 时钟线	建议通过 5kΩ 电阻上拉到 3.3V

```
src > C main.c > ...
110 static const char* html_script = "<script src=\"/web_script.js\"></script></body></html>";
111
112 // I2C 配置
113 #define I2C_MASTER_SCL_IO GPIO_NUM_5 /*!< GPIO number used for I2C master clock */
114 #define I2C_MASTER_SDA_IO GPIO_NUM_4 /*!< GPIO number used for I2C master data */
115 #define I2C_MASTER_NUM I2C_NUM_0 /*!< I2C port number for master dev */
116 #define I2C_MASTER_FREQ_HZ 400000 /*!< I2C master clock frequency (400kHz) */
117 #define I2C_MASTER_TX_BUF_DISABLE 0 /*!< I2C master doesn't need buffer */
118 #define I2C_MASTER_RX_BUF_DISABLE 0 /*!< I2C master doesn't need buffer */
119
120 // MLX90640 配置
121 const uint8_t MLX90640_I2C_ADDR = 0x33; // MLX90640 默认I2C地址
122 #define TA_SHIFT 8 // 来自库示例的环境温度偏移
123 static paramsMLX90640 mlx90640_params;
124 static float mlx90640_temperatures[THERMAL_DATA_SIZE];
125 static uint16_t eeMLX90640[832]; // EEPROM数据缓冲区
126 static bool mlx90640_initialized = false;
127
128 // 添加新的数据缓冲区和任务控制变量
129 static uint16_t mlx90640Frame[834]; // 静态以避免栈溢出
130 static SemaphoreHandle_t frameMutex = NULL;
131 static TaskHandle_t frameCaptureTaskHandle = NULL;
132 static bool taskRunning = false;
133 static float emissivity = 0.95;
134 static int mlx90640_init_status = -999;
135 static int mlx90640_eeprom_status = -999;
136 static int mlx90640_extract_status = -999;
137 static int mlx90640_refresh_status = -999;
138 static int mlx90640_chess_status = -999;
139 static int last_frame_status = 0;
140 static uint32_t frame_ok_count = 0;
141 static uint32_t frame_fail_count = 0;
142 static esp_err_t last_i2c_error = ESP_OK;
143 static uint32_t i2c_error_count = 0;
144 static uint8_t last_i2c_error_addr = 0;
145 static uint16_t last_i2c_error_reg = 0;
```

图 2 MLX90640 与 ESP32-C3 的 I2C 连接说明

上拉电阻与去耦电容的作用

I2C 的 SDA/SCL 属于开漏或开集电极风格信号，需要上拉电阻把空闲状态维持为高电平；10uF 电容并联在 VDD 与 GND 之间，用于降低供电波动。对于 MLX90640 这种阵列传感器，稳定供电和可靠接触会直接影响 I2C 扫描和帧数据读取。

4. 开发环境与工程结构

软件开发使用 VS Code + PlatformIO。PlatformIO 的作用不仅是编译代码，还包括管理 ESP-IDF 框架、开发板参数、串口烧录、串口监视、SPIFFS 文件系统镜像构建和网页资源上传。这样可以把 ESP32-C3 固件和网页前端放在同一个工程中统一管理。

```
1 ; PlatformIO Project Configuration File
2 ;
3 ; Build options: build flags, source filter
4 ; Upload options: custom upload port, speed and extra flags
5 ; Library options: dependencies, extra library storages
6 ; Advanced options: extra scripting
7 ;
8 ; Please visit documentation for the other options and examples
9 ; https://docs.platformio.org/page/projectconf.html
10
11 [env:esp32-c3-devkitm-1]
12 platform = espressif32@6.9.0
13 board = esp32-c3-devkitm-1
14 framework = espidf
15 monitor_speed = 115200
16 upload_speed = 921600
17 monitor_filters = direct
18 board_build.flash_size = 4MB
19 board_build.partitions = partitions.csv
20 board_build.filesystem = spiffs
21
22 ; 添加自定义上传目标
23 extra_scripts = pre:extra_script.py
24
```

图 3 VS Code 与 PlatformIO 工程环境

4.1 仓库关键文件

```
esp32c3-mlx90640-thermal-camera/
|-- platformio.ini          # PlatformIO 工程配置
|-- src/main.c              # ESP32-C3 主程序
|-- data/web_script.js      # 网页前端脚本
|-- lib/MLX90640/           # MLX90640 API 与算法库
|-- tools/mock_thermal_server.py # 本地演示服务器
|-- partitions.csv          # Flash 分区与 SPIFFS 配置
|-- README.md               # 运行方式与调试说明
`-- deliverables/           # 报告、PPT、图片和生成脚本
```

4.2 基本编译与烧录流程

```
pio run -e esp32-c3-devkitm-1
pio run -e esp32-c3-devkitm-1 -t upload
pio run -e esp32-c3-devkitm-1 -t uploadfs
pio device monitor -e esp32-c3-devkitm-1
```


其中 `upload` 用于烧录主程序，`uploadfs` 用于上传 SPIFFS 网页资源。实际调试中，如果烧录停在 `Connecting...`，需要按住 BOOT/FLASH 键，再轻按 RESET/RST，待出现写入信息后松开 BOOT/FLASH。

5. 软件架构与关键实现

软件部分围绕两条链路展开：第一条是传感器数据链路，即 I2C 初始化、MLX90640 参数读取、温度帧计算和共享缓存；第二条是网页数据链路，即 ESP32-C3 创建 WiFi 热点、启动 HTTP Server、返回 JSON 数据、浏览器周期性刷新并绘制热图。两条链路相互独立又通过温度缓存连接，便于在硬件不稳定时使用本地演示数据验证网页逻辑。

模块	主要职责	实现要点
I2C 驱动	配置 GPIO4/GPIO5、400kHz 时钟、读写寄存器	封装 MLX90640_I2CRead/Write 供 API 调用
传感器初始化	读取 EEPROM、提取校准参数、设置刷新率	记录初始化状态，方便串口和网页调试
采集任务	循环读取帧数据并计算 768 点温度	使用缓冲区和互斥量降低并发访问风险
HTTP 接口	向网页返回温度 JSON、状态和 I2C 扫描结果	便于判断硬件层、总线层和数据层问题
前端网页	绘制 Canvas 热图、色温条、CSV 导出	支持开始检测、导入数据、全屏和清空记录

```

src > C main.c > ...
264 // MLX90640 传感器初始化
265 static esp_err_t mlx90640_sensor_init(void) {
266     int status;
267     mlx90640_initialized = false;
268     mlx90640_init_status = -1;
269
270     status = MLX90640_DumpEE(MLX90640_I2C_ADDR, eeMLX90640);
271     mlx90640_eeprom_status = status;
272     if (status != 0) {
273         ESP_LOGE(TAG, "Failed to load EEPROM (status = %d)", status);
274         mlx90640_init_status = status;
275         return ESP_FAIL;
276     }
277     ESP_LOGI(TAG, "EEPROM loaded successfully.");
278
279     status = MLX90640_ExtractParameters(eeMLX90640, &mlx90640_params);
280     mlx90640_extract_status = status;
281     if (status != 0) {
282         ESP_LOGE(TAG, "Failed to extract parameters (status = %d)", status);
283         mlx90640_init_status = status;
284         return ESP_FAIL;
285     }
286     ESP_LOGI(TAG, "Parameters extracted successfully.");
287
288     // 提高刷新率到8Hz (0x04) 或 16Hz (0x05)
289     status = MLX90640_SetRefreshRate(MLX90640_I2C_ADDR, 0x05); // 16Hz
290     mlx90640_refresh_status = status;
291     if (status != 0) {
292         ESP_LOGE(TAG, "Failed to set refresh rate (status = %d)", status);
293         // 继续，即使刷新率设置失败
294     } else {
295         ESP_LOGI(TAG, "Refresh rate set to 16Hz.");
296     }
297
298     // 设置为棋盘模式 (Chess mode)
299     status = MLX90640_SetChessMode(MLX90640_I2C_ADDR);

```

图 4 MLX90640 初始化与参数读取代码截图

```

319 // 新增：持续捕获帧的任务
320 static void frame_capture_task(void *pvParameters) {
321     float tr; // 反射温度
322
323     while (taskRunning) {
324         // 获取帧数据
325         int status = MLX90640_GetFrameData(MLX90640_I2C_ADDR, mlx90640Frame);
326         last_frame_status = status;
327         if (status < 0) {
328             frame_fail_count++;
329             ESP_LOGE(TAG, "GetFrameData failed with status: %d", status);
330             vTaskDelay(pdMS_TO_TICKS(10));
331             continue;
332         }
333         frame_ok_count++;
334
335         float ta = MLX90640_GetTa(mlx90640Frame, &mlx90640_params);
336         tr = ta - TA_SHIFT; // 根据环境温度计算反射温度
337
338         // 获取互斥锁并更新温度数据
339         if (xSemaphoreTake(frameMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
340             MLX90640_CalculateTo(mlx90640Frame, &mlx90640_params, emissivity, tr, mlx90640_temperatures);
341             xSemaphoreGive(frameMutex);
342         }
343
344         // 不需要两帧间等待太久，传感器本身会按照配置的刷新率工作
345         vTaskDelay(pdMS_TO_TICKS(10)); // 短暂延迟以避免任务占用过多CPU
346     }
347
348     vTaskDelete(NULL);
349 }
350

```

图 5 温度帧采集任务代码截图

```

535 // HTTP GET处理函数 - 热成像数据
536 static esp_err_t thermal_data_get_handler(httpd_req_t *req)
537 {
538     if (!mlx90640_initialized) {
539         ESP_LOGE(TAG, "MLX90640 not initialized yet.");
540         httpd_resp_send_500(req);
541         return ESP_FAIL;
542     }
543
544     // 确保帧捕获任务正在运行
545     if (!taskRunning) {
546         start_frame_capture_task();
547     }
548
549     // 分配JSON缓冲区 - 使用静态缓冲区以避免频繁内存分配
550     static char json_buffer[JSON_BUFFER_SIZE];
551
552     // 从最新的温度数据生成JSON
553     if (xSemaphoreTake(frameMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
554         int current_pos = 0;
555
556         // 使用更快的方式构建JSON
557         json_buffer[current_pos++] = '[';
558
559         for(int i = 0; i < THERMAL_DATA_SIZE; i++) {
560             // 使用1位小数来减少数据大小，同时保持足够的精度
561             int written = snprintf(json_buffer + current_pos, JSON_BUFFER_SIZE - current_pos,
562                                   "%s%.1f", i == 0 ? "" : ",", mlx90640_temperatures[i]);
563
564             if (written < 0 || written >= JSON_BUFFER_SIZE - current_pos) {
565                 ESP_LOGE(TAG, "JSON buffer overflow");
566                 xSemaphoreGive(frameMutex);
567                 httpd_resp_send_500(req);
568                 return ESP_FAIL;
569             }
570             current_pos += written;

```

图 6 HTTP 温度数据接口代码截图

6. WiFi Web 服务与网页可视化

ESP32-C3 在 AP 模式下可以自己建立 WiFi 热点。手机或电脑连接该热点后，实际上进入了 ESP32-C3 建立的局域网，浏览器访问 `http://192.168.4.1` 时，请求会被发送到 ESP32-C3 内部运行的 HTTP Server。若使用 STA 模式连接路由器，则浏览器访问的是路由器分配给 ESP32-C3 的局域网 IP。

这些地址属于局域网私有地址，不是公网地址，因此手机切换到蜂窝网络或其他 WiFi 后无法直接访问。这是网络结构原因，不是网页代码错误。若后续需要远程访问，可以通过云端服务器、中转服务、内网穿透或 MQTT/HTTP 上报等方式把数据转发到公网可访问位置。


```

src > C main.c > ...
470 // WiFi初始化函数 - AP模式
471 void wifi_init_softap(void)
472 {
473     s_wifi_event_group = xEventGroupCreate();
474
475     ESP_ERROR_CHECK(esp_netif_init());
476     ESP_ERROR_CHECK(esp_event_loop_create_default());
477     esp_netif_t *ap_netif = esp_netif_create_default_wifi_ap();
478
479     wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
480     ESP_ERROR_CHECK(esp_wifi_init(&cfg));
481
482     ESP_ERROR_CHECK(esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));
483
484     // 获取MAC地址
485     uint8_t mac[6];
486     ESP_ERROR_CHECK(esp_wifi_get_mac(WIFI_IF_AP, mac));
487
488     // 生成SSID
489     char ssid[32];
490     snprintf(ssid, sizeof(ssid), "IRCAM-%02X%02X", mac[4], mac[5]);
491
492     wifi_config_t wifi_config = {
493         .ap = {
494             .max_connection = EXAMPLE_MAX_STA_CONN,
495             .authmode = WIFI_AUTH_OPEN,
496             .channel = 1,
497         },
498     };
499
500     // 复制SSID到配置结构
501     strcpy((char*)wifi_config.ap.ssid, ssid, sizeof(wifi_config.ap.ssid));
502     wifi_config.ap.ssid_len = strlen(ssid);
503
504     ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
505     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &wifi_config));
506     ESP_ERROR_CHECK(esp_wifi_start());

```

图 7 ESP32-C3 热点、局域网访问与网页服务关系

6.1 网页功能设计

- 开始检测：从 ESP32-C3 的 `/thermal-data` 接口读取真实传感器数据。
- 导入数据：使用网页端平滑演示数据，用于硬件不稳定或录制演示时验证可视化链路。
- 下载 CSV 数据：按照时间顺序导出已记录的温度帧，便于后续分析。
- 色温条与温度范围：显示当前最低温、最高温和颜色映射关系。
- 全屏展示：方便课堂答辩或演示视频录制。

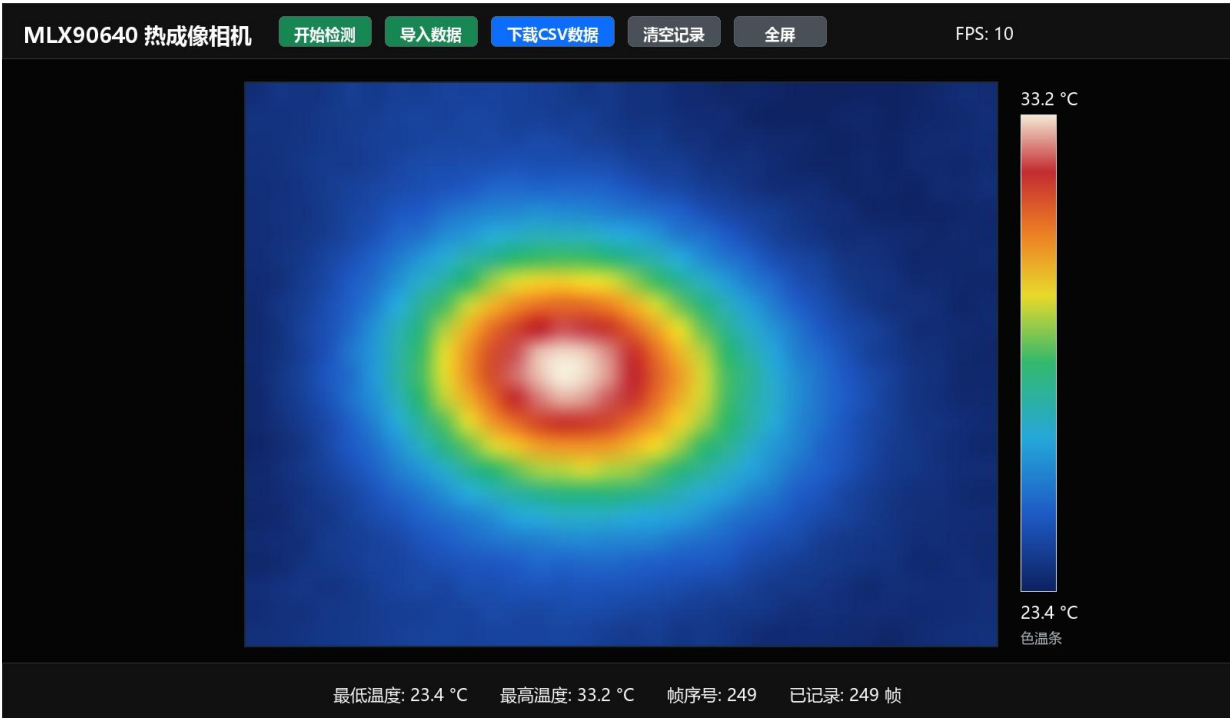


图 8 网页热图、色温条、数据记录 and 交互按钮效果

7. 调试过程与实验结果

项目调试遵循“先软件、再通信、后传感器”的思路。首先确认 PlatformIO 工程能编译，ESP32-C3 能被电脑识别为串口设备，主程序和 SPIFFS 网页资源能够烧录；随后通过浏览器确认网页能够打开；最后再进入 I2C 扫描、MLX90640 初始化和温度帧读取验证。

7.1 三层问题定位方法

层级	判断方法	期望现象	异常时重点检查
硬件层	看供电、串口、3.3V/GND	ESP32-C3 上电，电脑识别 COM 口	Type-C 数据线、GND 共地、杜邦线松动
总线层	I2C scan 或串口日志	扫描到 MLX90640 地址 0x33	SDA/SCL 是否接反，上拉是否有效
数据层	网页 min/max/热图变化	温度随热源移动变化	初始化状态、帧读取状态、JSON 接口

7.2 当前验证结果

- PlatformIO 工程可以完成主固件编译，ESP32-C3 固件和 SPIFFS 网页文件系统均已验证过构建流程。
- 网页端可以实现热图显示、色温条、开始检测、导入数据、全屏展示、清空记录和 CSV 下载等功能。
- 本地 Python 演示服务器能够生成平滑移动的热源数据，用于验证网页端热图和数据交互逻辑。

- 真实 MLX90640 采集链路已完成接线方案和软件接口，但受裸传感器、面包板、杜邦线松动影响，连续采集稳定性仍在验证中。

关于实地验证的说明

在课堂实地验证要求下，本项目已经完成实物搭建、固件烧录、网页访问和演示数据展示。由于 MLX90640 裸传感器与面包板连接不够稳定，真实温度帧在现场无法长时间稳定输出，因此演示时采用前期实验视频和网页端演示数据补充说明系统流程。该问题主要集中在硬件连接稳定性，不是 Web 界面、CSV 导出或 ESP32-C3 网页服务逻辑错误。

8. 问题分析与改进方向

从系统完整性看，项目的软件链路已经较完整：ESP32-C3 可以作为服务器提供网页和数据接口，网页端也能够完成热图、色温条和记录功能。现阶段最大的限制是硬件层可靠性，尤其是裸 MLX90640 引脚接触、面包板空间不足、杜邦线松动和 I2C 信号质量。要把系统从演示样机推进到稳定产品，需要优先解决硬件固定和信号完整性。

8.1 硬件优化

- 使用 MLX90640 转接板或自行设计 PCB，避免裸引脚直接插线导致的接触不良。
- 缩短 SDA/SCL 线长，保留 5k Ω 上拉电阻和 10uF 去耦电容，必要时重新评估 I2C 速率。
- 加入外壳和传感器固定结构，保证镜头方向、接线位置和演示时的机械稳定性。

8.2 算法与校准优化

- 加入异常点滤波、帧间平滑和热点追踪，使 32 x 24 低分辨率数据在网页端显示更自然。
- 建立温度校准流程，例如使用已知温度参考源做单点或两点校准，对偏移和增益进行补偿。
- 根据被测物体材料调整发射率参数，提高不同表面材料下的温度估计一致性。

8.3 功能拓展

- 加入温度极值触发 LED 或蜂鸣器报警，用于演示阈值检测和自动响应。
- 通过云端服务器、内网穿透或数据上报接口，使网页访问不局限于 ESP32-C3 局域网。
- 结合红外识别算法，扩展为电路板发热检测、设备异常识别或红外目标自动检测装置。

9. 总结

本项目完成了基于 ESP32-C3 与 MLX90640 的热成像相机系统设计与工程整理。系统将 I2C 红外阵列采集、ESP32-C3 WiFi Web 服务、浏览器热图可视化和 CSV 数据记录整合到同一工程中，体现了微机接口

技术从底层硬件连接到上层应用展示的完整流程。虽然真实传感器连续采集还受到临时接线稳定性的限制，但项目已经形成可编译、可烧录、可网页演示、可记录数据、可开源提交的完整软件与文档体系。

后续若完成传感器转接板或 PCB 固定，并加入校准、滤波、报警和远程访问功能，该系统可以进一步发展
为面向实验教学、电路板发热检测或红外异常识别的小型热成像平台。

附录：提交材料

材料	位置或说明
完整源码	src、data、lib、tools、platformio.ini、partitions.csv
运行说明	README.md，包含接线、编译、烧录、网页访问和调试接口
项目报告	deliverables/基于 ESP32-C3 与 MLX90640 的热成像相机系统项目报告.pdf
开发流程文档	deliverables/基于 ESP32-C3 与 MLX90640 的热成像相机系统开发流程.pdf
GitHub 仓库	https://github.com/1909899270h-spec/esp32c3-mlx90640-thermal-camera